AuraSpend Android Native Java Migration

Complete Technical Project Report & Java Implementation Guide

Platform: Android (Java 17, Min SDK 26, Target SDK 35) | Engine: Standalone SQLite

1. Executive Summary

AuraSpend is a personal finance and expense tracking Android application written in native Java with XML layouts and an embedded standalone SQLite database. The project was successfully transitioned from an earlier prototype requiring a PC-hosted Spring Boot and MySQL server to a 100% self-contained, offline-first mobile application.

Key Achievements:

- 100% Standalone Operation: Requires zero external servers, zero cloud dependency, and zero PC-hosted database. All data resides securely on the device.
- Multi-User Support: Multiple user accounts can register and authenticate on the same phone with isolated data and SHA-256 hashed passwords.
- Custom UI & Boot Animation: Crafted luxury obsidian-purple theme (#090414), 3D squircle app logo, and a choreographed startup boot animation with halo pulse.
- Full Feature Parity: Dashboard budget progress ring, searchable history, 3x2 category sheets, on-device PDF export, 9 currencies, and JSON backup/restore.

2. System Architecture

The application implements a clean layered MVC architecture tailored for Android:

- Presentation Layer: SplashActivity.java (boot animation), MainActivity.java (view orchestration), CircularProgressView.java (custom Canvas 2D ring), BottomSheetDialogs, and Material Components.
- Domain & Model Layer: User.java, Expense.java, AppSettings.java, Category.java (6 categories with emojis), and Currency.java (9 currencies with symbol formatting).
- Data Layer: DatabaseHelper.java (SQLiteOpenHelper managing auraspend.db) and SessionManager.java (SharedPreferences persistent session store).
- Security & Export: Android FileProvider providing secure URI grants for on-device PDF report exports and JSON backups.

3. Standalone SQLite Database Architecture

The database is embedded inside the Android app private storage at /data/data/com.auraspend.app/databases/auraspend.db.

Relational Database Schema (3 Tables):

Table Name	Primary Key	Key Columns
users	id (AUTOINCREMENT)	name, email (UNIQUE), password_hash, cur
expenses	id (AUTOINCREMENT)	user_email (FK), amount, date, category, des
settings	user_email (PK)	currency, budget, theme

Database Implementation Details (DatabaseHelper.java):

- Thread-Safe Singleton: DatabaseHelper.getInstance(context) prevents connection leaks and database lock contention.
- SHA-256 Cryptographic Hashing: Passwords are encrypted with SHA-256 message digests prior to storage; plaintext is never retained.

- Multi-User Isolation: All queries for expenses and settings strictly enforce WHERE user_email = ? clauses, isolating each user's finances.
- Performance Indexing: Indexes idx_expenses_user and idx_expenses_date guarantee sub-millisecond lookups for history and reports.
- Atomic Transactions: importBackup() wraps batch insertions in beginTransaction() and setTransactionSuccessful(), guaranteeing ACID integrity.

4. Screen-by-Screen Java Code Walkthrough

4.1 Boot-Up Splash Screen (SplashActivity.java)

SplashActivity serves as the application's launcher activity, orchestrating a multi-stage startup sequence before transitioning to MainActivity:

- Stage 1 (0ms): Ambient background glow orbs (#090414) fade in.
- Stage 2 (0-750ms): The 3D squircle logo springs in from scale 0.5 to 1.0 using an OvershootInterpolator(1.4f) for a tactile bounce.
- Stage 3 (100-800ms): A radial halo expands behind the logo and transitions into an infinite ValueAnimator breathing pulse.
- Stage 4 (300-900ms): The brand title 'AuraSpend' and subtitle 'Smart money, simple life.' slide up 40px and fade in with DecelerateInterpolator.
- Stage 5 (1850ms): Smooth crossfade transition into MainActivity with activity finish() so users cannot back-navigate to the splash screen.

4.2 Authentication Flow (MainActivity.java)

Authentication manages local login and registration without network reliance:

- Tab Switching: Toggles between loginForm and signupForm with visual chip highlights.
- Login: Queries dbHelper.loginUser(email, password). If valid, stores session in SessionManager and triggers enterApp().
- Sign Up: Validates name length, email format, and password length (min 6 chars). Calls dbHelper.registerUser(), which creates the user and seeds default settings.
- Session Persistence: On startup, SessionManager checks SharedPreferences for an active email. If found, auto-logs into the Dashboard.

4.3 Dashboard & Canvas Budget Ring

The dashboard aggregates real-time spending insights for the current calendar month:

- Hero Card: Displays monthly spending total, remaining budget, and the budget progress ring.
- CircularProgressView.java: Custom Android Canvas 2D view rendering the background track, progress arc, percentage, and 'budget' subtext.
- Stat Cards: 3 compact cards displaying total monthly transactions, daily average spending, and top expense category.
- Filter Chips: Interactive horizontally scrollable chips (All, Food, Transport, Rent, Fun, Utilities, Others) to filter category breakdown.
- Category Breakdown: Proportional progress bars scaled to the highest spending category with emoji badges.

4.4 Searchable History & Activity

The history screen provides chronological transaction logs with real-time search:

- Real-Time Search: TextWatcher monitors user keystrokes in searchBox, filtering both descriptions and category names simultaneously.
- Date Grouping: Transactions are bucketed by date (YYYY-MM-DD), rendering human-readable headers (e.g. 'Thursday, Oct 29') and daily subtotals.
- Tap to Edit: Clicking any transaction card opens the Edit Expense bottom sheet pre-filled

with the item’s existing data.

4.5 Add & Edit Expense Bottom Sheets

Interactive slide-up bottom sheets with Material BottomSheetDialog:

- Category Grid: 3x2 interactive tile grid (Food, Transport, Rent, Fun, Utilities, Others) with selected state highlights.
- Dynamic Currency Prefix: Automatically displays the user’s active currency symbol ($ / Rs. / E / L) next to the amount input.
- Native DatePicker: Tapping the date field opens Android’s native calendar DatePickerDialog defaulting to the current date.
- Deletion Flow: Edit sheet provides a danger-styled ’Delete transaction’ button with an AlertDialog confirmation prompt.

4.6 Reports & On-Device PDF Engine

Comprehensive financial insights with native PDF generation:

- Period Tabs: Weekly (Monday-current), Monthly (current month), Annual (current year), and All-time.
- Summary Card: Displays timeframe total spending and transaction count.
- Native PDF Generator: Built using Android’s native android.graphics.pdf.PdfDocument API. Generates clean multi-page tabular reports directly on the device.
- Sharing: Opens the Android Share Sheet via FileProvider with temporary URI read permissions.

4.7 Settings, Appearance & Data Portability

User preferences, theme customization, and data backup:

- Currency Selector: Modal chooser for 9 currencies (USD, INR, EUR, GBP, AED, CAD, AUD, JPY, SGD), updating all screens immediately.
- Budget Adjustment: Dialog to adjust monthly budget threshold, updating the budget ring.
- Appearance Toggle: Instant toggle between Dark Obsidian and Light Lavender themes, persisted in SQLite.
- JSON Export/Import: Full backup compatibility with AuraSpend v2 backups. Imports validate JSON structure and transactionally insert records.

5. Security, Build Quality & Verification

Quality assurance and security measures:

- Data Privacy: Internal app storage sandboxing prevents cross-app access without device root.
- Password Protection: SHA-256 one-way cryptographic hashing protects credentials.
- FileProvider Security: Content URIs with FLAG_GRANT_READ_URI_PERMISSION avoid broad external storage permissions.
- Compilation: Verified with Gradle 8.9 and JDK 17 (BUILD SUCCESSFUL, app-debug.apk generated).
- Automated Unit Tests: CurrencyAndCategoryTest passes 100% of test assertions for calculations, currencies, and hashing.
- Rootless Inspection: Full compatibility with Android Studio Database Inspector and Termux Wireless ADB.